TP2: Grover

Foreword

Welcome back. If TP1 taught you how to encode functions as oracles, TP2 is about turning those oracles into a *search engine*. Given an unstructured list of N items, a classical computer needs $O(N)$ tries to find the one you are looking for. Grover's algorithm does it in $O(\sqrt{N})$, which is a provably optimal speedup for this kind of problem, and one of the most beautiful uses of quantum interference ever designed. Schrödinger's cat would be proud: not only is the answer simultaneously found and not found, but we actively encourage it to be both until the very last moment, and then collapse it in our favor.

The whole thing boils down to a geometric trick. Your uniform superposition lives in a 2D plane spanned by the "good" states (the ones you are looking for) and the "bad" states (everything else). Each Grover iteration rotates your state inside that plane, a little closer to the good states each time. After about $\frac{\pi}{4}\sqrt{N}$ iterations, you are practically sitting on top of the solution, and a single measurement reveals it with high probability. Appendix A explains the whole thing with a circle and some high-school trigonometry. If you have never seen Grover before, reading it now will save you a lot of head-scratching later.

A few survival tips:

- **Iteration count matters.** Too few iterations and you miss; too many and you actually rotate *past* the solution. Grover is not monotonic. Unlike gradient descent, more work does *not* mean a better answer. It is the only algorithm where your professor is legally allowed to tell you to stop trying.
- **Reuse TP1.** The phase oracle you wrote last time is the exact same pattern you need here. No-cloning theorem says you cannot copy a qubit, but it says nothing about copying your own code.
- **Endianness strikes again.** Q#'s little-endian convention is about to bite you once more, especially in the multi-element version where secrets come as `Bool[]` arrays. If your results look uniformly scrambled, blame endianness before blaming the multiverse.
- **When in doubt,** `DumpMachine()`**.** Watching amplitudes grow on marked states, iteration after iteration, is honestly one of the most satisfying things you can see in a quantum simulator. It is the closest thing quantum computing has to a progress bar.

Each exercise below comes with a **Hint** box to unblock you and a **Test** box to verify your implementation. Good luck, and may your amplitudes concentrate where you want them to (and nowhere Heisenberg would complain about).

1 Single Grover

These exercises are found in the `SingleGrover.qs` file.

Grover's algorithm is a quantum search algorithm that finds a marked element in an unstructured list with a complexity of $O(\sqrt{N})$, whereas a classical algorithm would require $O(N)$ operations.

*If you want to understand **why** the algorithm works (the geometric picture, where the $\frac{\pi}{4}\sqrt{N}$ comes from, and why running it too long actually makes things worse), head to Appendix A first.*

1.1 Oracle

Let's begin with a phase oracle.

For this oracle, we will mark the state $|x\rangle$:

$$U_f |\psi\rangle = \begin{cases} -|\psi\rangle & \text{if } |\psi\rangle = |x\rangle \\ |\psi\rangle & \text{otherwise} \end{cases}$$

which is equivalent to

$$U_f |\psi\rangle = (-1)^{f(\psi)} |\psi\rangle \quad \text{with} \quad \begin{cases} f(\psi) = 1 & \text{if } |\psi\rangle = |x\rangle \\ f(\psi) = 0 & \text{otherwise} \end{cases}$$

Complete the `GenerateGroverPhaseOracle(secret: Int): (Qubit[] => Unit)` function for this exercise.

You are allowed to create helper functions to implement this oracle. For example, you may create a marking oracle function first and then use it to implement the phase oracle. A marking oracle applies a bit flip to an auxiliary qubit when the input state matches the target state.

If you encounter difficulties implementing this function, please refer to TP1 where similar oracle implementations were discussed.

Hint. This is structurally identical to the `PhaseOracle` from TP1: the only difference is that the marked state is now a parameter rather than hardcoded to $|101\rangle$.

Notice the return type: `(Qubit[] => Unit)`. This is a *function that returns an operation.* The cleanest way in Q# is to return a lambda that captures `secret` in its closure:

```
function GenerateGroverPhaseOracle(secret: Int)
    : (Qubit[] => Unit) {
    return (qs) => {
        use aux = Qubit();
        within { X(aux); H(aux); }
        apply { /* mark here */ }
    };
}
```

Inside the `apply` block, reuse `ApplyControlledOnInt(secret, X, qs, aux)` from TP1. The `within` prepares $|-\rangle$ on the ancilla, the oracle kicks back the phase, and Q# decomputes the ancilla automatically.

Test. With `secret = 5` and 3 qubits, prepare a uniform superposition with `ApplyToEach(H, qs)`, call the oracle, and `DumpMachine()`. You should see seven amplitudes at $+1/\sqrt{8}$ and exactly one at $-1/\sqrt{8}$, on the basis state at index 5 (which is $|101\rangle$ in little-endian). If the minus sign lands on the wrong index, you probably passed the secret in the wrong byte order.

1.2 Grover Diffusion Operator

The diffusion operator is the crucial part of Grover's algorithm. It can be expressed as:

$$D = 2\left|s\right\rangle\left\langle s\right| - I$$

where $\left|s\right\rangle = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}\left|x\right\rangle$ is a uniform superposition of all basis states, and I is the identity operator.

The diffusion operator performs a reflection around the uniform superposition, which amplifies the amplitude of the marked state after each iteration of the algorithm. See Appendix A for a pictorial explanation of why this reflection is exactly the right move.

In practice, the diffusion operator can be implemented as follows:

- Apply $H^{\otimes n}$ to all qubits
- Apply the operation $2\left|0\right\rangle\left\langle 0\right| - I$ (a reflection around $\left|0\right\rangle$)
- Apply $H^{\otimes n}$ to all qubits again

While this description is correct, implementing the reflection around $\left|0\right\rangle$ directly can be complex. A more practical approach is to decompose this operation into elementary gates as follows:

$$D = H^{\otimes n} \cdot (2\left|0\right\rangle\left\langle 0\right| - I) \cdot H^{\otimes n}$$

The reflection around $\left|0\right\rangle$, $2\left|0\right\rangle\left\langle 0\right| - I$, can be implemented as:

$$2\left|0\right\rangle\left\langle 0\right| - I = X^{\otimes n} \cdot (I - 2\left|1\right\rangle\left\langle 1\right|) \cdot X^{\otimes n} = X^{\otimes n} \cdot \mathrm{CZ} \cdot X^{\otimes n}$$

where CZ is a multi-controlled-Z operation with all qubits as controls except the last, which serves as target.

Therefore, the diffusion operator can be implemented with the sequence:

$$D = H^{\otimes n} \cdot X^{\otimes n} \cdot \mathrm{CZ} \cdot X^{\otimes n} \cdot H^{\otimes n}$$

Complete the `GroverDiffusionOperator(qs: Qubit[]): Unit` function to implement the diffusion operator.

Hint. The operator is symmetric: `H`s outside, `X`s just inside, `CZ` in the middle. You can express the two outer layers in a single `within` block, since Q# will automatically replay them in reverse order as the adjoint:

```
within {
    ApplyToEachA(H, qs);
    ApplyToEachA(X, qs);
} apply {
    // multi-controlled Z here
}
```

For the multi-controlled Z, use `Controlled Z(controls, target)` from the standard library. Rather than slicing `qs` by hand with `qs[0..Length(qs) - 2]`, the `Std.Arrays` functions `Most` (all elements except the last) and `Tail` (the last element) read much better.

Test. The diffusion operator leaves the uniform superposition $\left|s\right\rangle$ untouched (it is its own fixed point, since $D\left|s\right\rangle = \left|s\right\rangle$). Run `ApplyToEach(H, qs)` then `GroverDiffusionOperator(qs)` and `DumpMachine()`: you should get back a perfectly uniform superposition with all amplitudes at $+1/\sqrt{2^n}$. If any amplitude is negative or if magnitudes differ, one of the `within` blocks is missing its `A` suffix or the multi-controlled Z is wired incorrectly.

1.3 Grover

Finally, we implement Grover.

The complete Grover's algorithm can be described by the following steps:

- Initialize n qubits in the state $|0\rangle^{\otimes n}$
- Apply Hadamard gates to all qubits to create a uniform superposition: $H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}|x\rangle$
- Repeat the following operation r times:
  - Apply the phase oracle U_ω
  - Apply the diffusion operator D

  The optimal number of iterations is $r \approx \frac{\pi}{4}\sqrt{N}$.
- Measure the n qubits

For $n = 5$ qubits, the optimal number of iterations is $r \approx \frac{\pi}{4}\sqrt{32} \approx 4.4$, so we will use $r = 5$ iterations (we round up the result). The derivation of this formula is in Appendix A.

Complete the `Grover(secret: Int, nbQubits: Int, nbIterations: Int): Result[]` function to implement the complete Grover's algorithm with 5 iterations, using the phase oracle and diffusion operator you have implemented previously. Your function should return the measurement results after running the algorithm.

Hint. The structure is: initial `H` layer, then a `for` loop applying oracle then diffusion, then measure. Something like:

```
use qs = Qubit[nbQubits];
let oracle = GenerateGroverPhaseOracle(secret);
ApplyToEach(H, qs);
for _ in 1..nbIterations {
    oracle(qs);
    GroverDiffusionOperator(qs);
}
return MResetEachZ(qs);
```

Note that the underscore in `for _ in ...` means "I do not care about the index". Q# accepts it, and it makes the intent clear.

Test. Run `Grover(5, 3, 2)` many times (say 100 runs from a Python or Q# loop) and count how often it returns $|101\rangle$, that is, `[One, Zero, One]`. For $n = 3$, the optimal number of iterations is $\lceil \frac{\pi}{4}\sqrt{8} \rceil = 2$, and the success probability should be close to 1 (around 94%). Try `nbIterations = 1` to see a lower success rate, and `nbIterations = 10` to observe the amplitude rotating *past* the solution and coming back down. Grover is periodic, not monotonic.

2 Multiple Grover

These exercises are found in the `MultipleGrover.qs` file.

The Multiple Grover algorithm extends the standard Grover search to find multiple marked elements in an unstructured database. The circuit is unchanged, only the oracle and the iteration count are adjusted.

2.1 Oracle for Multiple Elements

Let's begin with a phase oracle that marks multiple states.

For this oracle, we will mark the states $|x_1\rangle, |x_2\rangle, \ldots, |x_m\rangle$ where m is the number of marked elements:

$$U_f |\psi\rangle = \begin{cases} -|\psi\rangle & \text{if } |\psi\rangle \in \{|x_1\rangle, |x_2\rangle, \ldots, |x_m\rangle\} \\ |\psi\rangle & \text{otherwise} \end{cases}$$

which is equivalent to

$$U_f |\psi\rangle = (-1)^{f(\psi)} |\psi\rangle \quad \text{with} \quad \begin{cases} f(\psi) = 1 & \text{if } |\psi\rangle \in \{|x_1\rangle, \ldots, |x_m\rangle\} \\ f(\psi) = 0 & \text{otherwise} \end{cases}$$

Complete the `GenerateGroverPhaseOracle(secrets: Bool[][]): (Qubit[] => Unit)` function for this exercise.

Note that each element of the `secrets` array is a bit string (represented as a `Bool` array), and the function should mark all states corresponding to any of these bit strings. You are allowed to create helper functions to implement this oracle.

Hint. Since phases compose multiplicatively, marking a set of states is the same as marking each state in sequence inside the same `within ... apply` block. Allocate *one* ancilla in $|-\rangle$, then loop over `secrets` and apply one controlled flip per entry. The ancilla accumulates the product of all kickbacks and gets decomputed only once at the end:

For the per-element flip, `ApplyControlledOnBitString(s, X, qs, aux)` is the direct `Bool[]` analogue of `ApplyControlledOnInt`: it flips `aux` when `qs` matches the exact bit pattern in `s`. The overall shape is the same lambda-returning-closure pattern as for Single Grover, with a `for s in secrets` loop inside the `apply` block.

Test. For 3 qubits, mark two states: `secrets = [[true, false, true], [false, true, false]]`, which corresponds to $|101\rangle$ and $|010\rangle$ in little-endian, namely indices 5 and 2. After `ApplyToEach(H, qs)` and the oracle, `DumpMachine()` should show six amplitudes at $+1/\sqrt{8}$ and two at $-1/\sqrt{8}$, on indices 2 and 5. If only one amplitude is negative, your loop only handles the first secret; if all are negative, the ancilla is not being decomputed between iterations.

2.2 Grover Diffusion Operator

The Grover diffusion operator for Multiple Grover is exactly the same as for Single Grover.

The diffusion operator is defined as:

$$D = 2 |s\rangle \langle s| - I$$

where $|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$ is a uniform superposition of all basis states, and I is the identity operator.

This operator is completely independent of which states are marked by the oracle, as it always performs a reflection around the uniform superposition. Whether we have a single marked state or multiple marked states, the diffusion operator remains unchanged.

Therefore, you can import and reuse the exact same `GroverDiffusionOperator` function that you implemented for Single Grover:

```
import SingleGrover.GroverDiffusionOperator;
```

2.3 Optimal Number of Iterations

With multiple marked elements, we need to calculate the optimal number of iterations in Grover's algorithm.

The optimal number of iterations is given by:

$$r \approx \frac{\pi}{4}\sqrt{\frac{N}{M}}$$

where $N = 2^n$ is the size of the search space for n qubits, and M is the number of marked elements (the length of the `secrets` array). Appendix A explains where the $\sqrt{N/M}$ comes from geometrically.

Implement the `getNbIterations(secrets: Bool[][], nbQubits: Int): Int` function to calculate the optimal number of iterations for a given number of qubits and marked elements. You should use the formula above and round up to the nearest integer using the `Ceiling` function.

Hint. You will need four helpers from the standard library: `IntAsDouble` (from `Std.Convert`), `Sqrt`, `Ceiling` and `PI` (all from `Std.Math`). The computation is a one-liner once the types line up.

A couple of details: $N = 2^n$ can be written as `1 <<< nbQubits` (bit shift on an `Int`), which avoids importing a power function; and remember that the division `N / M` is an `Int`/`Int` division, so cast its result to a `Double` with `IntAsDouble` *before* passing it to `Sqrt`.

Test. A few values to check by hand: with $n = 5$ and $M = 1$, you should get $\lceil \pi/4 \cdot \sqrt{32} \rceil = 5$. With $n = 5$ and $M = 4$, you should get $\lceil \pi/4 \cdot \sqrt{8} \rceil = 3$. With $n = 10$ and $M = 1$, you should get 26. If your function returns 0 for small cases, you are probably doing integer division before casting to double.

2.4 Multiple Grover

The implementation consists of two functions, where the main one calls the other:

- `GroverWithIterations(secrets: Bool[][], nbQubits: Int, nbIterations: Int): Result[]`: implements the actual Grover algorithm with a specified number of iterations.
- `Grover(secrets: Bool[][], nbQubits: Int): Result[]`: simply calculates the optimal number of iterations and calls `GroverWithIterations`.

The main implementation for `GroverWithIterations` follows these steps:

- Initialize n qubits in the state $|0\rangle^{\otimes n}$
- Apply Hadamard gates to all qubits to create a uniform superposition: $H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}|x\rangle$
- Repeat the following operations r times (where r is the number of iterations provided):
  - Apply the phase oracle U_f that marks multiple states
  - Apply the diffusion operator D (imported from `SingleGrover`)
- Measure all qubits and return the results

For the `Grover` function, you simply need to:

- Calculate the optimal number of iterations using your `getNbIterations` function
- Call your `GroverWithIterations` function with this optimal number
- Return the results from `GroverWithIterations`

This approach allows you to focus on implementing the core algorithm once in `GroverWithIterations`, while `Grover` just handles the calculation of the optimal number of iterations.

Hint. `GroverWithIterations` is a near-copy of your Single Grover circuit, with the oracle now generated from a `Bool[][]` instead of an `Int`. `Grover` is a two-line wrapper: compute `nbIterations`, call `GroverWithIterations`.
The point of splitting into two functions is so that the Python analysis section can drive `GroverWithIterations` directly and sweep the iteration count, which is exactly what you need to plot accuracy versus number of iterations.

Test. With `secrets = [[true, false, true], [false, true, false]]` and `nbQubits = 3`, your `Grover` should return either $|101\rangle$ or $|010\rangle$ (both with roughly equal probability) on most runs. If you run it 100 times, the two marked states combined should account for more than 90% of the outcomes. A tell-tale sign of a broken oracle is getting $|000\rangle$ or other unmarked states too often. A tell-tale sign of a wrong iteration count is getting roughly uniform outcomes across all 2^n basis states.

3 Grover analysis (Python)

*This part of the subject is made harder and with less guidance, on purpose. You **need** to have done the other exercises beforehand.*

You will need to create a Python notebook file which will use the functions implemented before in the Q# part of your work. Then, you will do an analysis of multiple aspects of the Grover algorithm, such as:

- Plotting the accuracy versus the number of iterations
- Benchmarking it
- Plotting the accuracy versus the number of valid states
- And much more, feel free to try anything

For this, you will be provided with a `uv` environment as in *TP 0*, where you will need to run `uv sync` to generate the virtual environment. It notably contains `numpy`, `matplotlib` and `qsharp`. If you add other packages, let us know and modify the `uv` environment according to your changes.

Hint. The `qsharp` Python package lets you import your Q# operations as if they were Python functions. Point it at your project root and import directly:

```
import numpy as np
import qsharp
qsharp.init(project_root='.')
from qsharp.code.MultipleGrover import GroverWithIterations
```

From there, `GroverWithIterations(secrets, nbQubits, nbIterations)` behaves like any other Python function: call it in a loop a few hundred times, count how often the result is a marked state, and you have an empirical success probability.

For the accuracy-versus-iterations plot, sweep `nbIterations` from 0 to something like $2\,r_{\text{opt}}$, run each value a few hundred times, and plot the fraction of runs that returned a marked state. You should see a sine-squared curve peaking near r_{opt}. That is Grover's geometric rotation (see Appendix A), drawn directly in probability space.

Test. Two sanity checks for your notebook:

- The accuracy-versus-iterations curve should hit its first maximum at $r \approx \frac{\pi}{4}\sqrt{N/M}$ and then oscillate. If it grows monotonically, you are measuring something other than success probability.
- The accuracy-versus-M plot (varying the number of marked elements at fixed N and r_{opt}) should stay close to 1. If it collapses as M grows, you are probably not recomputing r_{opt} for each M.

A How Grover actually works (the friendly version)

Good news: Grover is one of the few quantum algorithms that you can genuinely understand with nothing more than high-school trigonometry and a good mental image of a clock. No tensor products, no fancy identities, no staring at Hilbert spaces until they stare back. Just a circle, an angle, and some patience.

The story is this: we have a mystery item hidden somewhere among N possibilities, and the quantum computer is going to gradually "point" at it, one rotation at a time, until a measurement reveals it. Everything else is details.

Step 1: sorting items into two boxes

We are looking through a list of $N = 2^n$ items. Among them, M are the "good ones" (marked by our oracle; for Single Grover, just $M = 1$). Everything else is "bad".

Classically, finding a good one is boring: you pick items at random and check them one by one. On average, you try N/M times before getting lucky. For $N = 32$ items with $M = 1$ good one, that is 32 tries in the worst case.

Quantum computing is going to do something sneakier. Instead of looking at items one at a time, we put the computer in a superposition of all of them at once, and then *tilt* that superposition so it leans toward the good ones. At the end, we measure, and (if we timed it right) we almost certainly get a good item.

To keep things manageable, we group all the good items into one "bundle" and all the bad items into another bundle:

$$|G\rangle = \text{“a random good item”}, \qquad |B\rangle = \text{“a random bad item”}$$

Formally:

$$|G\rangle = \frac{1}{\sqrt{M}} \sum_{x \text{ marked}} |x\rangle\,, \qquad |B\rangle = \frac{1}{\sqrt{N-M}} \sum_{x \text{ not marked}} |x\rangle$$

The $1/\sqrt{M}$ and $1/\sqrt{N-M}$ are just normalization factors so that each bundle has length 1 (quantum states must have total probability 1). You can ignore those details for intuition.

Why this matters. The quantum state Grover produces is always some mixture of $|G\rangle$ and $|B\rangle$. It never escapes this two-bundle world. Why not? Because both Grover operators respect this structure: the oracle only distinguishes "good" from "bad" (so it treats all good states identically and all bad states identically), and the diffusion operator only depends on the uniform superposition $|s\rangle$, which itself lives in the $(|G\rangle\,, |B\rangle)$ plane. Neither operator has any way of singling out one particular good state versus another, so they can only move our arrow within the plane, never out of it. This is pure symmetry, and it is what reduces a huge problem to a drawing on a sheet of paper.

Step 2: drawing the state on a circle

Let us put $|B\rangle$ on the horizontal axis and $|G\rangle$ on the vertical axis, like x and y. Because quantum states have length 1, our state is a point on the unit circle.

The state at the start, after a layer of Hadamard gates, is the uniform superposition:

$$|s\rangle = \frac{1}{\sqrt{N}}\big(\,|x_1\rangle + |x_2\rangle + \cdots + |x_N\rangle\,\big)$$

Group the good ones with the good ones, and the bad ones with the bad ones:

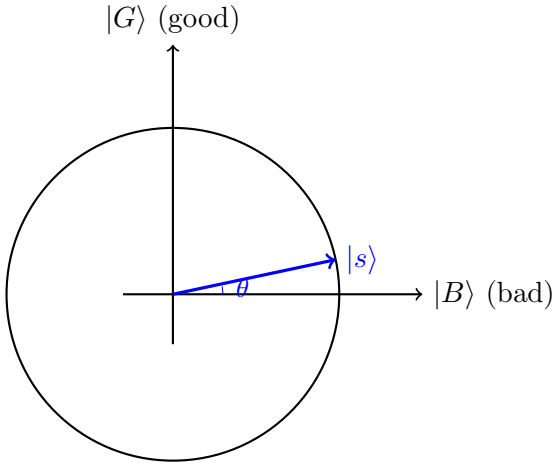
$$|s\rangle = \sqrt{\tfrac{M}{N}}\,|G\rangle + \sqrt{\tfrac{N-M}{N}}\,|B\rangle$$

The coefficients are exactly what a classical probability computation would predict: the chance of grabbing a good item at random is M/N, and in quantum mechanics, the *amplitude* is the square root of the probability.

Now, any point on the unit circle can be written with an angle θ measured from the horizontal axis:

$$|s\rangle = \cos\theta \cdot |B\rangle + \sin\theta \cdot |G\rangle$$

Comparing with the formula above: $\sin\theta = \sqrt{M/N}$. For $N = 32$ and $M = 1$, that gives $\sin\theta = 1/\sqrt{32} \approx 0.177$, so $\theta \approx 10°$. The state leans almost entirely toward $|B\rangle$, with only a tiny tilt toward $|G\rangle$. Makes sense: the good item is rare, so the superposition is dominated by bad items.



If we measured the state right now, the probability of landing on a good item would be the square of the vertical component:

$$P(\text{good}) = \sin^2\theta = \frac{M}{N}$$

That is the same probability as guessing randomly. So far, the quantum computer has done nothing impressive. This is where Grover kicks in.

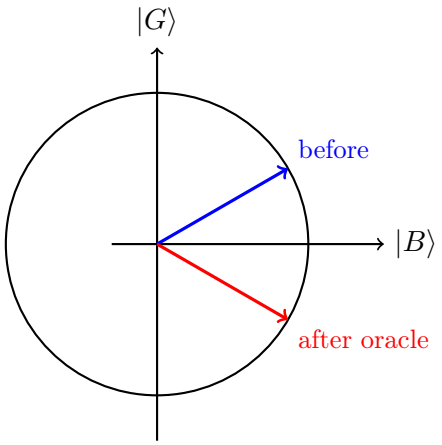
The whole goal of the algorithm: rotate the little blue arrow all the way up to $|G\rangle$ (straight up), so that measuring is guaranteed to give a good item.

Step 3: the oracle flips the arrow across the floor

The oracle from Section 1 multiplies good states by -1 and leaves bad states alone. That minus sign is a *phase flip*, not a probability change: if you measured right after the oracle, the probability of landing on a good item would still be M/N, because $|-1|^2 = 1$. Nothing observable has happened yet. What changed is that we now have a *marked* direction in our 2D picture, and the next step (diffusion) will exploit that mark to tilt the arrow in our favor.

If your arrow is at angle α above the horizontal, its vertical component is $\sin\alpha$. After the oracle, that vertical component becomes $-\sin\alpha$. The horizontal component is untouched. In other words, the arrow ends up at angle $-\alpha$: below the horizontal, mirror-imaged.

This is a **reflection across the horizontal axis** (across the "floor" of our picture, the $|B\rangle$ axis). Nothing mysterious; you did this kind of thing in geometry class with folded paper.



Step 4: the diffusion operator flips across $|s\rangle$

The diffusion operator $D = 2|s\rangle\langle s| - I$ does the same kind of thing, but the mirror is different: it reflects across the line going through $|s\rangle$ (the diagonal line where the original superposition lives).

Why? Two quick checks are enough to convince yourself:

- Apply D to $|s\rangle$ itself: $D|s\rangle = 2|s\rangle \cdot 1 - |s\rangle = |s\rangle$. The state on the mirror line does not move. ✓
- Apply D to any vector $|v\rangle$ perpendicular to $|s\rangle$: $D|v\rangle = 2|s\rangle \cdot 0 - |v\rangle = -|v\rangle$. Perpendicular vectors get flipped. ✓

"Leaves the mirror line alone, flips perpendicular directions" is exactly what a reflection does.

Step 5: the clincher, two flips make a rotation

Here is the geometric fact that makes Grover tick, straight from high-school geometry:

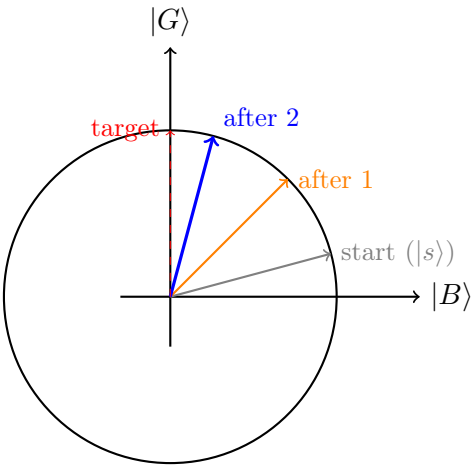
> *If you reflect a point across one line, then across a second line, the combined effect is the same as rotating the point. The rotation angle is twice the angle between the two lines.*

If you are skeptical, grab a piece of paper, draw two lines meeting at some angle β, pick a point, reflect it across the first line, then across the second. Measure the angle between the original point and the final one. It will be 2β.

In Grover's case:

- First mirror: the horizontal axis $|B\rangle$ (the oracle).
- Second mirror: the line through $|s\rangle$ (the diffusion operator).
- Angle between them: θ (since $|s\rangle$ makes angle θ with the horizontal).

So **one full Grover iteration (oracle then diffusion) rotates the arrow by** 2θ. And crucially, it rotates *upward*, toward $|G\rangle$.



After k iterations, the arrow has moved a total of $2k\theta$ beyond the starting angle θ, ending up at $(2k+1)\theta$ from the horizontal. The probability of measuring a good item is the square of the vertical component:

$$P_k(\text{good}) = \sin^2\big((2k+1)\theta\big)$$

Step 6: finding the sweet spot

We want the arrow to land right on $|G\rangle$, meaning its angle to the horizontal should be exactly $90° = \pi/2$ radians. So we want:

$$(2k+1)\theta \;=\; \frac{\pi}{2}$$

Solving for k:

$$k_{\text{opt}} = \frac{\pi}{4\theta} - \frac{1}{2}$$

Now, when $M \ll N$ (rare good items, the typical case), the angle θ is small, and there is a useful approximation from trigonometry: $\sin\theta \approx \theta$ when θ is small and *measured in radians*. This approximation breaks down if θ is not small (for example when M gets close to $N/2$), or if you forget the radians and use degrees. As long as we are in the standard regime $M \ll N$, we are safe. Since $\sin\theta = \sqrt{M/N}$, we get $\theta \approx \sqrt{M/N}$. Plugging in:

$$\boxed{k_{\text{opt}} \;\approx\; \frac{\pi}{4}\sqrt{\frac{N}{M}}}$$

This is **the** Grover formula. Since k has to be a whole number, we pick the integer *closest* to k_{opt} (not always up) because that is the one that lands our arrow closest to $|G\rangle$ and maximizes the success probability. In practice, for the sizes used in this TP, rounding up (via `Ceiling`) and rounding to nearest usually give the same integer, which is why the subject uses `Ceiling`. But if you are ever optimizing at the edge, prefer the true nearest integer.

Worked examples (and a sanity check for the subject):

- $N = 32$, $M = 1$: $k_{\text{opt}} \approx \frac{\pi}{4}\sqrt{32} \approx 4.44$, so we use $k = 5$. Exactly the number used in Section 1.3.
- $N = 32$, $M = 4$: $k_{\text{opt}} \approx \frac{\pi}{4}\sqrt{8} \approx 2.22$, so we use $k = 3$. More good items means fewer iterations, which makes sense: the more there are, the easier they are to find.

Step 7: why pushing harder makes things worse

The success probability is $\sin^2\big((2k+1)\theta\big)$. Because $\sin^2$ is periodic, things get weird if you keep iterating past k_{opt}:

- At $k = k_{\text{opt}}$: angle is $\pi/2$, probability is $\sin^2(\pi/2) = 1$. Perfect.
- At $k = 2k_{\text{opt}}$: angle is around $3\pi/2$, probability is back near zero. The arrow has rotated past $|G\rangle$ and come back down around the circle.
- At $k = 3k_{\text{opt}}$: probability climbs back up to near 1 again.

The accuracy plot oscillates like a sine wave, forever.

This is Grover's defining quirk: *more effort is not always better.* Classical search rewards patience, Grover rewards precision. Running Grover too long is like overshooting the exit on a roundabout and having to go all the way around again. You will see this exact wave in the Python analysis section when you plot accuracy versus iteration count.

Step 8: the speedup, summarized on a napkin

Classical search needs $O(N/M)$ queries in the worst case. Grover needs about $\frac{\pi}{4}\sqrt{N/M}$. Why $\sqrt{N/M}$ and not N/M?

Look again at what each iteration does: it rotates the arrow by a fixed angle $2\theta \approx 2\sqrt{M/N}$. We need to cover $\pi/2$ radians total to get from $|s\rangle$ to $|G\rangle$. So the number of steps is

$$\text{number of iterations} = \frac{\text{total angle to cover}}{\text{angle per step}} = \frac{\pi/2}{2\sqrt{M/N}} = \frac{\pi}{4}\sqrt{\frac{N}{M}}$$

The square root comes from a single fact: each iteration rotates by $\sqrt{M/N}$, not by M/N. That is the entire magic. Every time someone says "Grover gives a quadratic speedup", what they actually mean is "each iteration rotates, and the angle of rotation depends on the amplitude (square root of probability), not on the probability itself".

One-line takeaway

> *Grover is just a controlled rotation in a 2D plane, engineered by composing two reflections.*

If you remember only one sentence from this appendix, make it that one. Everything else (the oracle, the diffusion operator, the $\sqrt{N}$ speedup, the optimal iteration count, the periodicity) follows from it mechanically.